



NATIONAL ECONOMICS UNIVERSITY

FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

Data Analysis with Python

MIDTERM PROJECT REPORT

BOOK RATING CLASSIFICATION

Book Recommendation Dataset

Assignment 6 — Classification

Students

Lương Văn Thái Tuấn 11258665

Nguyễn Thị Khánh Hòa 11253156

Supporting lecturer

Dr. Trần Đức Minh

GitHub: https://github.com/Bluenip/Data_analyst_python_midterm_project

10 November 2026

Abstract

This report uses the Books, Users, and Ratings tables from the Book Recommendation Dataset to formulate a binary classification task at the individual-rating level. Only ratings from 1 to 10 are used as labels; ratings of 8 or higher are designated class 1. Zero-valued records are retained when describing rating activity but are not treated as low ratings. After integration and cleaning, the classification dataset contains **433,671 ratings**, of which **57.62%** belong to class 1.

The workflow covers key validation and encoding checks, ISBN normalization, missing and anomalous value handling, exploratory data analysis (EDA), feature construction, a stratified 70/15/15 train/validation/test split, and comparisons of four algorithms across five feature groups. Each algorithm uses the same configuration across feature groups. Imputation, scaling, and one-hot encoding are fit on training data only; user- and book-level rates are generated with smoothed out-of-fold (OOF) encoding to avoid using a row's own label to encode that row.

Of the 20 evaluated configurations, Logistic Regression with Target Encoding was selected by validation Macro F1. On the test set, it achieved an Accuracy of **65.65%**, a Macro F1 of **0.6553**, and a ROC-AUC of **0.7306**. Compared with a baseline that always predicts the majority class, Macro F1 increased by **0.2897** and Accuracy by **8.03 percentage points**. The results indicate that rating-history features from the training set provide predictive signal, but performance declines for cases with both an unseen user and an unseen book. The evaluation covers one random rating-level split; limitations are discussed in Section 9.2.

Contents

Abstract i

1 Problem Definition and Scope 1
1.1 Objective . . . 1
1.2 Research Questions . . . 1
1.3 Interpretation Scope . . . 1

2 Data, Validation, and Integration 2
2.1 Source and Structure . . . 2
2.2 Key and Initial Quality Checks . . . 2
2.3 Relationships and Join Strategy . . . 2

3 Cleaning and Classification Dataset Construction 3
3.1 Cleaning Rules . . . 3
3.2 Remaining Missing Values and Training-Time Handling . . . 4

4 Exploratory Data Analysis 4
4.1 Age and Class Distributions . . . 4
4.2 Rating Scores and Book Popularity . . . 5
4.3 Age and Rating Class . . . 6
4.4 User Activity and Rating Tendencies . . . 7
4.5 Correlation Matrix of Numeric Variables . . . 9

5 Feature Engineering and Data-Split Design 10
5.1 Five Cumulative Feature Groups . . . 10
5.2 Train, Validation, and Test Split . . . 11
5.3 Count Features and Target Encoding Without Self-Label Leakage . . . 11

6 Algorithms, Baseline, and Evaluation Criteria 12
6.1 Model Configurations . . . 12
6.2 Baseline . . . 12
6.3 Metrics and Selection Rule . . . 13

7 Validation Results and Model Selection 13
7.1 Comparison Across Feature Groups . . . 13
7.2 Effects of Count Features and Target Encoding . . . 14
7.3 Configuration Selected by Validation . . . 15

8 Test Evaluation and Generalization Scope 16
8.1 Selected Model Compared with the Baseline . . . 16
8.2 Confusion Matrix and Error Types . . . 17
8.3 ROC and Discrimination Across Thresholds . . . 18
8.4 Results by Seen and Unseen IDs . . . 18

9 Discussion, Applications, and Limitations 19
9.1 Interpretation of Findings . . . 19
9.2 Limitations to Consider . . . 19

10 Conclusion and Future Work 20
10.1 Conclusion . . . 20
10.2 Next Steps . . . 20

Reproducibility and Data Provenance 21

References 21

A Complete Validation Results for 20 Configurations 22

List of Figures

1 User age distribution after applying the 5–100 validity rule; 166,848 users with nonmissing ages. . . . 4
2 Class distribution among the 433,671 explicit ratings; each rating contributes once. 5
3 Counts of explicit ratings by score (1–10); zero-valued ratings are excluded. . . 5
4 Share of high ratings (8–10) by age group. . . . 7
5 User age versus explicit rating count, with a logarithmic rating-count axis. . . . 8
6 Comparison of mean user-level high-rating shares across activity groups. . . . 8
7 Spearman correlation matrix among numeric variables and the target `liked`; "—" marks undefined pairs. . . . 9
8 Validation Macro F1 by algorithm and feature group; the dashed line is the baseline. 14
9 Test confusion matrix; rows show true labels and columns show predicted labels. 17
10 ROC curve for the final model on the test set. . . . 18

List of Tables

1 Size of the three raw data tables. . . . 2
2 Missing data in the classification dataset after the clean join. . . . 4
3 Five most frequently rated books in Ratings, including zero-valued ratings. . . . 6
4 Ratings and class 1 share by age group, restricted to valid ages. . . . 6
5 Mean user-level high-rating share by activity group. . . . 8
6 Input feature groups compared in the experiments. . . . 10
7 Sizes of the stratified data splits. . . . 11
8 Configurations held fixed across the five feature groups. . . . 12
9 Validation Macro F1 for four algorithms across five input feature groups. . . . 13
10 Logistic Regression performance as input feature groups are added sequentially. 15
11 Validation metrics for the Target Encoding feature group. . . . 15
12 Test results for Logistic Regression + Target Encoding and the baseline. . . . 16
13 Test results by whether users and books appeared in the training set. . . . 18
14 Environment recorded in the notebook run used for this report. . . . 21
15 All validation metrics for five feature groups and four algorithms. . . . 22

1 Problem Definition and Scope

1.1 Objective

The unit of observation is one user's rating of one book. The objective is to estimate whether a rating belongs to the high-rating range. The task is framed as binary classification rather than exact score prediction:

$$y_i = \text{liked}_i = \begin{cases} 1, & \text{if } 8 \leq r_i \leq 10, \\ 0, & \text{if } 1 \leq r_i \leq 7. \end{cases} \tag{1}$$

Here, r_i denotes the `Book-Rating` for observation i. The threshold of 8 is a project convention, not a requirement of the midterm assignment. Class 0 means only that a rating does not meet the high-rating threshold; it does not imply that the user disliked the book.

According to the data provider, ratings from 1 to 10 are explicit, while 0 represents implicit feedback [1, 2]. Because zero-valued records do not provide an explicit preference score, they are not assigned to either class in Equation (1).

1.2 Research Questions

1. What patterns characterize the class distribution, user ages, and user activity levels?
2. Do book metadata, country, and ID-based statistics improve validation classification performance over the initial age- and year-based features?
3. Which configuration best identifies both classes and improves on the baseline?
4. Does aggregate performance reflect results for users or books unseen during training?

1.3 Interpretation Scope

The data represent only ratings present in the collected dataset. The model classifies a rating known to have a score from 1 to 10; it does not predict the probability that an arbitrary user will read, purchase, or rate a book. Classification metrics also do not replace ranking metrics for a recommendation system. Findings from EDA are descriptive or hypothesis-generating. The main sequence is data validation and cleaning, EDA, feature construction, validation-based selection, and test error analysis.

2 Data, Validation, and Integration

2.1 Source and Structure

The data come from Möbius's Book Recommendation Dataset on Kaggle, the source specified for Assignment 6 [1]. The three tables provide distinct types of information, summarized in Table 1.

Table 1: Size of the three raw data tables.

Table	Rows	Columns
Books	271,360	8
Users	278,858	3
Ratings	1,149,780	3

Source: Notebook, Section 1: Data Inspection.

- **Books:** ISBN, title, author, publication year, publisher, and three cover-image URLs.
- **Users:** User-ID, Location, and Age.
- **Ratings:** User-ID, ISBN, and Book-Rating.

Image URLs are excluded because the project does not process images. ISBNs are loaded as strings to preserve leading zeros and the character X. All three files are checked using strict UTF-8 decoding before being read with `pandas.read_csv`; a loop automates the loading process.

2.2 Key and Initial Quality Checks

In the raw data, ISBNs in Books and User-IDs in Users are unique. There are no exact duplicate rows or duplicate User-ID–ISBN pairs in Ratings. A total of 118,644 ratings have ISBNs that do not match Books; every User-ID in Ratings matches Users. The checks also find one Books ISBN and 14 Ratings ISBNs with leading or trailing whitespace. These join-key issues must be resolved before creating the clean analysis table, even though the original keys are unique.

Raw ages range from 0 to 244 and require review before use. Three book rows contain publisher names in the publication-year field, with some following fields containing misaligned URLs. These rows are isolated during cleaning rather than corrected by guesswork.

2.3 Relationships and Join Strategy

Ratings is the central table. Each user may contribute multiple ratings, and each book may appear in multiple ratings, so both relationships are many-to-one from Ratings to their respective lookup tables:

$$\texttt{Ratings.User-ID} \longrightarrow \texttt{Users.User-ID}$$
$$\texttt{Ratings.ISBN} \longrightarrow \texttt{Books.ISBN}.$$

The project uses a **left join**, not concatenation, because the three tables have different structures. The `validate='many_to_one'` parameter ensures that duplicate keys in a lookup table do not multiply rows; `indicator` records match status. After each join, the row count is checked against the number of Ratings records. Retaining records without book metadata avoids silently shrinking the sample; missing values are handled explicitly during preprocessing.

3 Cleaning and Classification Dataset Construction

3.1 Cleaning Rules

1. **ISBN and duplicates:** strip leading and trailing whitespace from ISBNs in Books and Ratings, then remove exact duplicate rows. One duplicate Books row is removed after normalization. If conflicting metadata remain for an ISBN or conflicting ratings for a User-ID–ISBN pair, the program stops for review rather than selecting a record automatically.
2. **Publication year:** convert values to numeric and isolate the three rows that cannot be converted because of field misalignment. Year 0 and years outside 1450–2026 are treated as missing. This is a project cleaning rule, not a claim that every value outside this range is invalid.
3. **Age:** retain ages from 5 to 100 and set other values to missing without deleting users. The number of users with missing age increases from 110,762 to 112,010 after this step.
4. **Strings:** normalize whitespace, decode HTML entities in book titles, and replace missing or empty author, publisher, and location values in the lookup tables with `Unknown`. Missing values introduced by the left join are retained and handled in the pipeline.
5. **Missingness indicators:** create `Age_missing` and `Year_missing`, then update them after joining to include books with no matching metadata. Median imputation is not performed at this stage.
6. **Ratings and target:** validate integer ratings in the range 0–10 and store them as a small integer type; use only ratings from 1 to 10 to create `liked`, as defined in Equation (1).

The clean joined table retains all 1,149,780 ratings. After key normalization and isolation of malformed rows, 118,645 ratings have no clean book metadata. This total is one higher than the raw unmatched-ISBN count because normalization and quarantining change which book keys are available. Within the 433,671-rating classification dataset, 49,829 records have no book metadata, or 11.49%. These counts use different denominators and should not be interchanged.

3.2 Remaining Missing Values and Training-Time Handling

Table 2: Missing data in the classification dataset after the clean join.

Field	Missing ratings	Percentage
Age	130,571	30.11%
Year-Of-Publication	55,635	12.83%
Book-Title	49,829	11.49%
Book-Author	49,829	11.49%
Publisher	49,829	11.49%

Source: Notebook, missing-value table in Section 3; denominator: 433,671 ratings.

Age is missing for 30.11% of classification ratings but for 40.17% of users in Users; the difference arises because one user may contribute multiple ratings. For numeric variables, the pipeline learns the median from training data for imputation and then scales the values. Missing categorical values are represented as `Unknown`. Missingness indicators allow the model to distinguish observed values from imputed ones.

4 Exploratory Data Analysis

4.1 Age and Class Distributions

After cleaning, 166,848 users have valid ages and 112,010 have missing ages. The mean known age is approximately 34.75 and the median is 32; the 25th and 75th percentiles are 24 and 44. Figure 1 summarizes users rather than rating records: each user contributes once. Most known ages are among younger and middle-aged adults; users with missing ages are excluded.

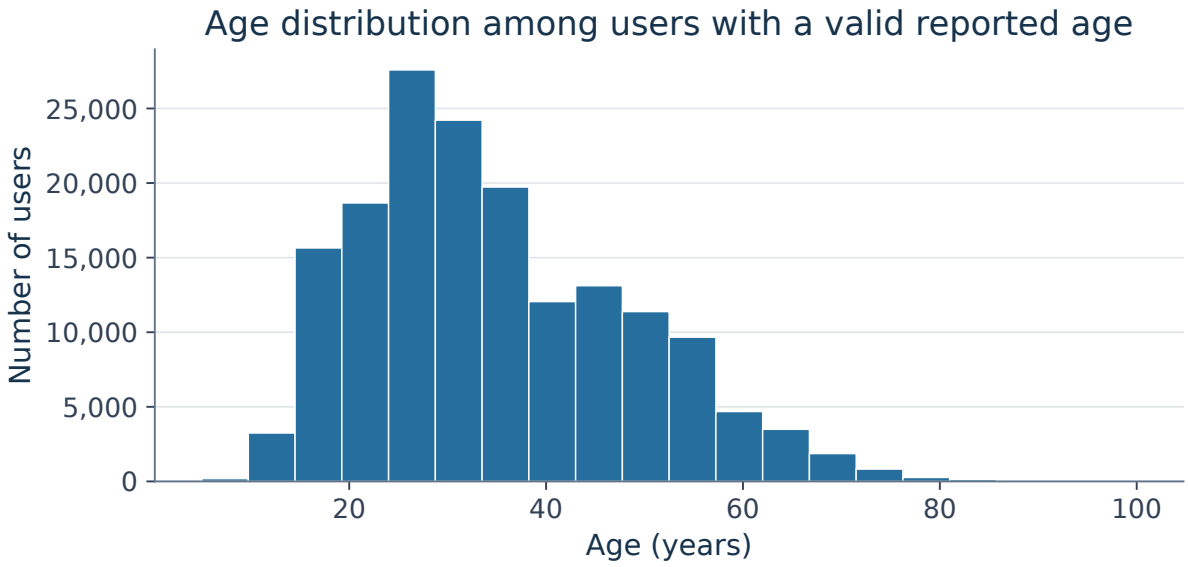


Figure 1: User age distribution after applying the 5–100 validity rule; 166,848 users with nonmissing ages.
Source: Notebook, Section 4.1; chart regenerated from Users.

Class 1 contains **249,887 ratings** (**57.62%**), while class 0 contains **183,784 ratings** (**42.38%**); see Figure 2. This is a modest class imbalance. Always predicting class 1 already yields about 57.62% Accuracy, so the evaluation also uses Macro F1 and per-class Recall to assess recognition of both groups.

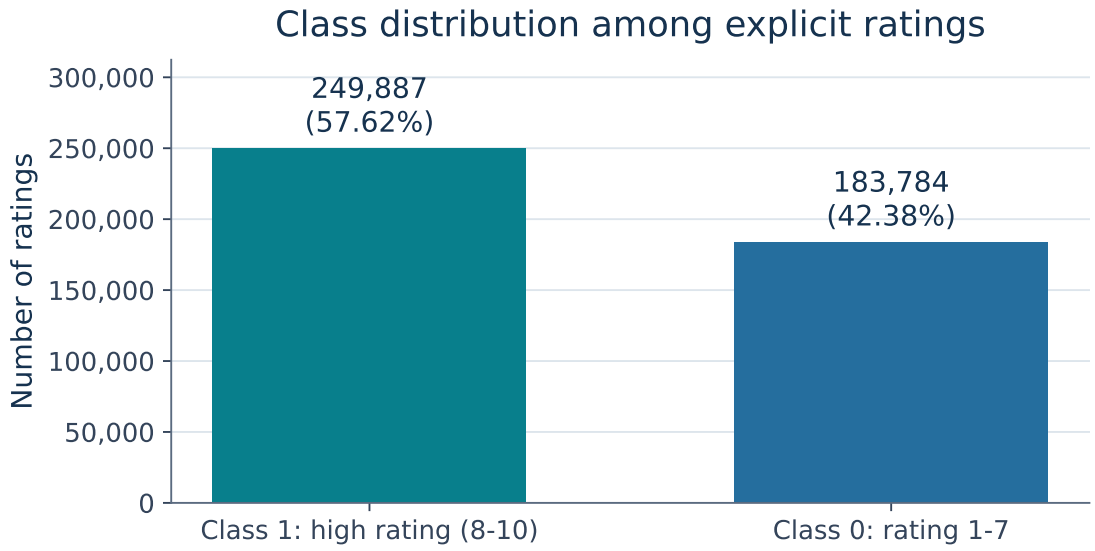


Figure 2: Class distribution among the 433,671 explicit ratings; each rating contributes once.
Source: Notebook, Section 4.2; chart regenerated from Ratings.

4.2 Rating Scores and Book Popularity

Of the 1,149,780 raw ratings, 716,109 have a score of 0 and 433,671 have scores from 1 to 10. Among explicit ratings, score 8 is most frequent with 103,736 ratings, followed by score 10 with 78,610 and score 7 with 76,457. The distribution is tilted toward relatively high scores, consistent with class 1 representing more than half of the classification dataset.

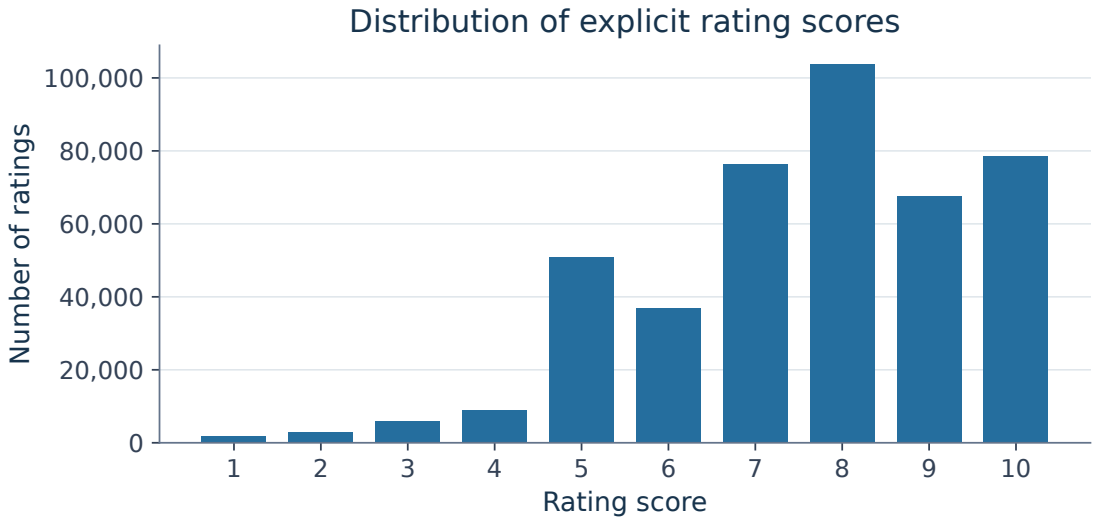


Figure 3: Counts of explicit ratings by score (1–10); zero-valued ratings are excluded.
Source: Notebook, Section 4.2.

Table 3: Five most frequently rated books in Ratings, including zero-valued ratings.

ISBN	Title	Ratings
0971880107	Wild Animus	2,502
0316666343	The Lovely Bones: A Novel	1,295
0385504209	The Da Vinci Code	883
0060928336	Divine Secrets of the Ya-Ya Sisterhood: A Novel	732
0312195516	The Red Tent (Bestselling Backlist)	723

Source: Notebook, Section 4.4.

Table 3 measures rating frequency, not preference. For example, *Wild Animus* ranks first with 2,502 ratings, but its mean explicit score is only about 4.39 across 581 rated records. Conversely, the book with ISBN 1888054557 has a mean score of 10.00 but only 11 explicit ratings. This illustrates why rating counts and mean scores should be considered together; it does not establish a negative correlation between popularity and scores across all books.

When ranking mean scores, the notebook includes only books with at least 10 explicit ratings to reduce the influence of books with very few scores. This threshold does not eliminate small-sample variation; rankings based on 10–20 ratings should be interpreted cautiously.

4.3 Age and Rating Class

Among ratings with a known user age, mean ages in classes 0 and 1 are approximately 35.99 and 36.78, respectively. The difference is descriptive; a comparison of means alone cannot establish how well age classifies individual ratings. When age is grouped into bins, the proportion of high ratings varies as shown in Figure 4 and Table 4.

Table 4: Ratings and class 1 share by age group, restricted to valid ages.

Age group	Ratings	High-rating share
5–14	3,875	60.28%
15–24	42,987	58.85%
25–34	105,871	58.57%
35–44	73,988	59.55%
45–54	51,458	64.93%
55–64	18,963	60.45%
65–74	4,774	59.61%
75–84	859	53.20%
85–94	75	34.67%
95–100	250	76.00%

Source: Notebook, Section 4.6.

The 45–54 age group has 51,458 ratings and a 64.93% high-rating share. The 85–94 group has a 34.67% share based on only 75 ratings, while the 95–100 group reaches 76.00% across

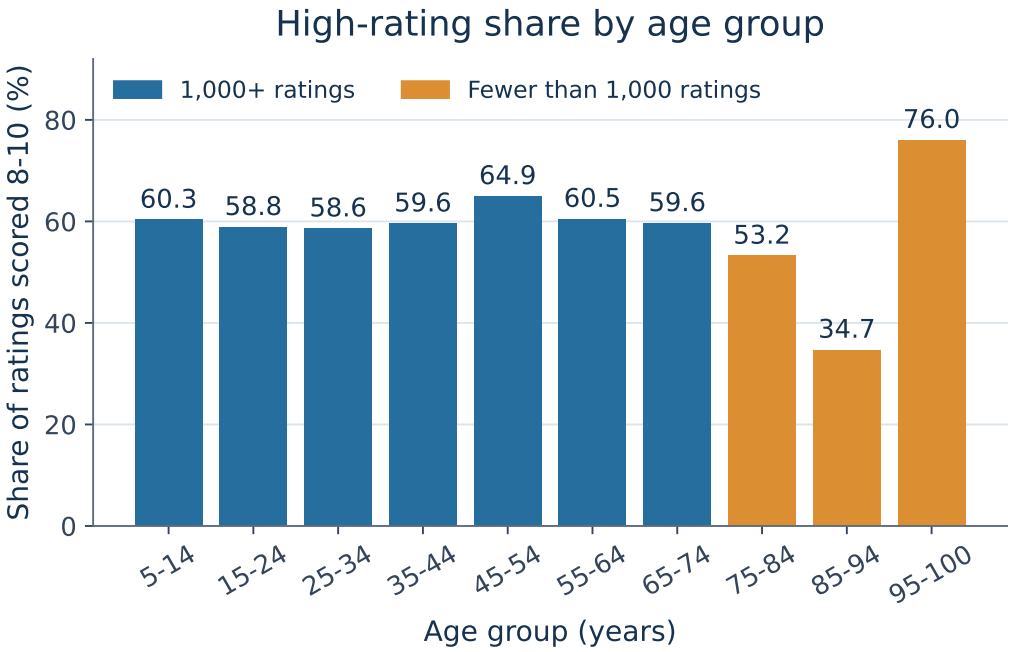


Figure 4: Share of high ratings (8–10) by age group.
Source: Notebook, Section 4.6.

250 ratings. Extremes in small groups should not be treated as stable patterns. Each row is a rating, so highly active users contribute more weight to these proportions.

For the 10×2 age-group/class table, the notebook reports $\chi^2(9) = 706.6158$ and $p \approx 2.61537 \times 10^{-146}$. As a descriptive effect-size summary, Cramér's $V = \sqrt{\chi^2/(N \min(9,1))} \approx 0.0483$, with $N = 303{,}100$ ratings, indicates a small association in this contingency table despite the tiny nominal p-value. The usual independence assumption may fail because users and books contribute repeated ratings; the p-value is therefore a reference calculation, not population-level confirmation. Age remains a candidate feature to be evaluated through validation performance.

4.4 User Activity and Rating Tendencies

The most active user by explicit ratings is ID 11676 with 8,524 ratings, followed by ID 98391 with 5,802. Most users have far fewer ratings. Figure 5 shows users with a valid known age and at least one explicit rating. Its logarithmic count axis makes the low-activity majority visible alongside highly active users; users with missing ages do not appear in this plot.

To avoid treating all ratings as independent users, the next analysis aggregates each user's class 1 share. The low-activity group contains users at or below the 25th percentile of rating counts (≤ 1 rating), while the high-activity group contains users at or above the 75th percentile (≥ 3 ratings). The two groups do not overlap in this dataset.

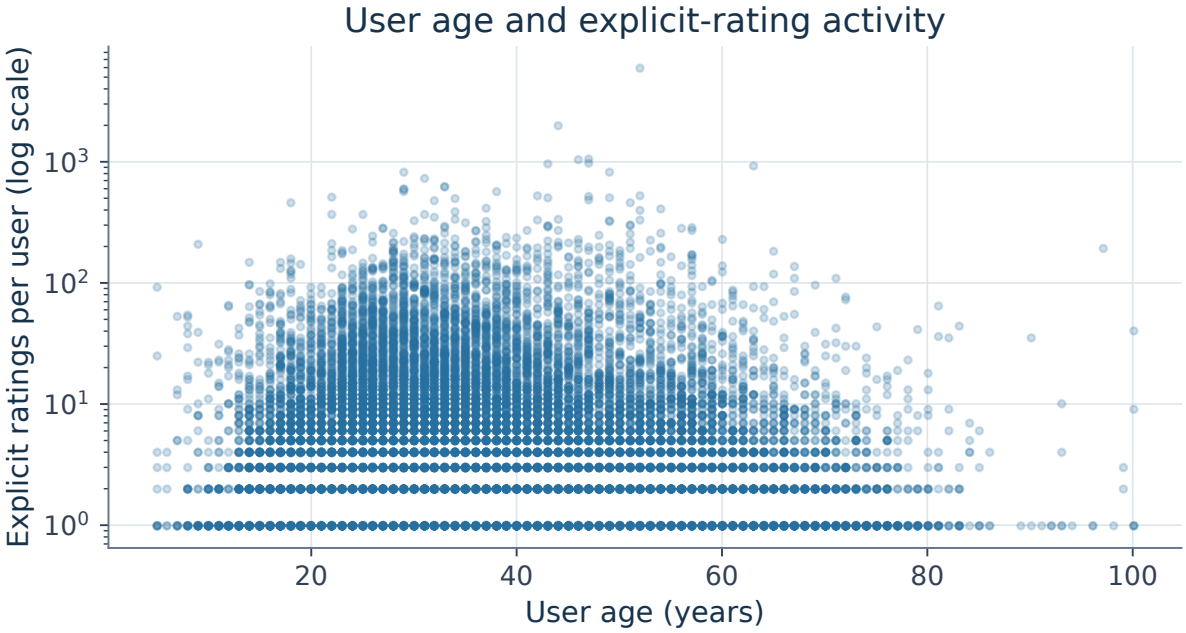


Figure 5: User age versus explicit rating count, with a logarithmic rating-count axis.

Source: Notebook, Section 4.7.

Table 5: Mean user-level high-rating share by activity group.

Group	Users	Median ratings per user	Mean user-level high-rating share
Low activity	45,382	1	56.27%
High activity	22,568	6	53.00%

Source: Notebook, Section 4.8.

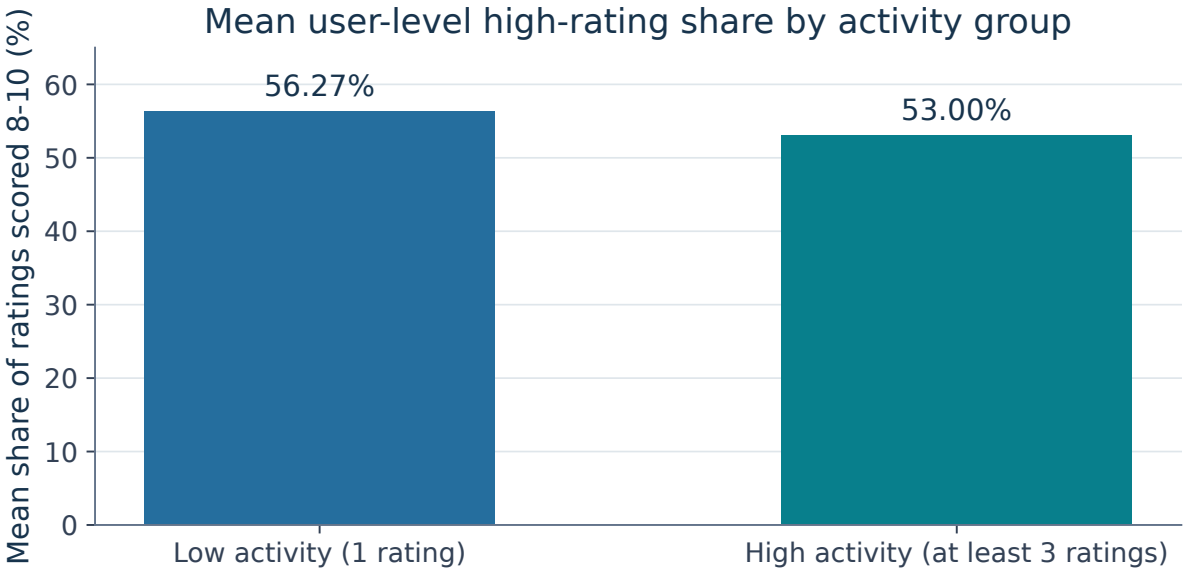


Figure 6: Comparison of mean user-level high-rating shares across activity groups.

Source: Notebook, Section 4.8.

Each user contributes one share to the group mean, regardless of their number of ratings; users with exactly two ratings belong to neither comparison group. The high-activity group has a mean high-rating share about 3.28 percentage points lower, using unrounded rates. The Mann–Whitney U statistic is $U = 566,384,643$, with $p \approx 3.21475 \times 10^{-125}$. However, a user with one rating always has a share of 0 or 1, while a user with many ratings can have a share between these extremes. Different rating counts therefore produce different variability, so a difference in distributions does not automatically demonstrate a difference in underlying preference tendencies. Users may also rate many of the same books.

These results motivate testing activity counts and ID-based statistics. Full-sample EDA summaries are not used directly as model inputs; features for modeling are recomputed from the training set.

4.5 Correlation Matrix of Numeric Variables

Figure 7 reports a Spearman correlation matrix for the numeric candidate inputs and the target over the 433,671 explicit ratings. Spearman's ρ is used instead of Pearson's because user and book rating counts are extremely right-skewed; a rank-based coefficient is less sensitive to the few very active users and very popular books. `Book-Rating` is excluded because `liked` is defined from it. The two count variables are computed on the full classification dataset for exploration only; the count features used for modeling are recomputed from the training set (Section 5.3). Missing values are dropped pairwise, so Age uses 303,100 ratings and publication year 378,036.

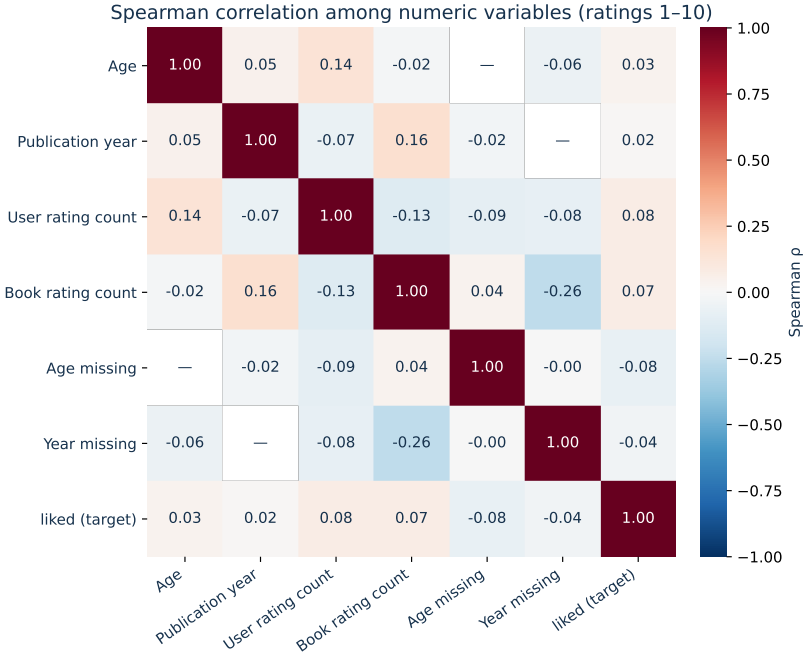


Figure 7: Spearman correlation matrix among numeric variables and the target `liked`; "—" marks undefined pairs.

Source: Notebook, Section 4.9.

All coefficients with `liked` are small: the largest in magnitude are Age missing ($\rho = -0.0757$), user rating count (0.0753), and book rating count (0.0717), while Age (0.0316) and publication year (0.0153) are weaker. No single numeric variable therefore separates the two classes on its own. Among the inputs, the strongest pair is book rating count and Year missing ($\rho = -0.2565$): books without a valid publication year tend to be rated less often. No input pair is strong enough to suggest serious multicollinearity. Cells marked "—" are undefined because a missing-value flag is constant whenever its source variable is observed.

These are univariate, monotonic associations. A weak coefficient does not rule out usefulness in combination with other features, and a stronger one does not imply causation. The matrix motivates keeping the missing-value flags and testing ID-based statistics, but feature groups are still selected by validation performance (Section 7).

5 Feature Engineering and Data-Split Design

5.1 Five Cumulative Feature Groups

Table 6 presents the experimental sequence. Column counts refer to inputs before one-hot encoding, not the number of dimensions after encoding. Each step adds only the listed group, while keeping each algorithm's configuration fixed.

Table 6: Input feature groups compared in the experiments.

Group	Features	Columns
Initial	Age, Age_missing, Year-Of-Publication, Year_missing	4
Metadata	Initial + Book-Author, Publisher	6
Country	Metadata + country parsed from Location	7
Count	Country + User_Rating_Count, Book_Rating_Count	9
Target Encoding	Count + User_Liked_Rate, Book_Liked_Rate	11

`Book-Rating` is excluded from X because the target is derived directly from that column. User-ID and ISBN are used only as keys for computing statistics, not as continuous numeric inputs. Book titles, full Location strings, and image URLs are not used directly. EDA does not establish the independent effects of publication year, author, or publisher; these variables are treated as candidates and evaluated through validation.

Country is parsed from the final comma-separated component of Location, then normalized to lowercase and trimmed. Missing markers such as empty strings, `n/a`, `none`, and `unknown` are mapped to `Unknown`; aliases such as `us`/`united states` are mapped to `usa`, and `uk` to `united kingdom`. Country is unknown for 20,551 ratings, or 4.74% of the classification dataset. This rule does not validate all geographic values or correct every self-reported error.

5.2 Train, Validation, and Test Split

Data are randomly split by rating, stratified by label, with `random_state=56`. First, 15% is held out for test; then $15/85$ of the remaining data is used for validation.

Table 7: Sizes of the stratified data splits.

Split	Ratings	Class 1 share
Train	303,569	57.62%
Validation	65,051	57.62%
Test	65,051	57.62%

Source: Notebook, Section 5.2.

Assertions verify that X and y have matching indices, that the three splits do not overlap, and that their union reconstructs the classification dataset. Imputation medians, scaling parameters, and infrequent-category groupings are fit on training data only in the pipeline, following recommended practices to prevent preprocessing leakage [4]. Categorical variables use one-hot encoding with `min_frequency=100`. Infrequent categories are grouped; a new value maps to that group if it exists for the feature, and otherwise produces an all-zero one-hot block.

5.3 Count Features and Target Encoding Without Self-Label Leakage

The two count features use the number of *ratings scored from 1 to 10 in the training set*. For a training row, one is subtracted from that ID's count to exclude the current rating. For validation and test, counts are looked up from training data only; unseen IDs receive 0. These values are not total book reads or total activity including zero-valued ratings.

For an ID e in fit subset G, the smoothed class rate is calculated as:

$$\mathrm{TE}_G(e) = \frac{s_G(e) + \alpha\mu_G}{n_G(e) + \alpha}, \qquad \mu_G = \frac{1}{|G|}\sum_{j\in G} y_j, \quad \alpha = 20, \tag{2}$$

where $n_G(e)$ is the number of ratings and $s_G(e)$ is the sum of class 1 labels for ID e in subset G. The training set is divided into five stratified folds with seed 42; each fold receives rates computed from the other four folds only. Neither the ID statistics nor the overall rate μ_G uses labels from the fold being encoded. For validation and test, G is the full training set; unseen IDs receive the training-set prior.

Smoothing limits extreme rates for IDs with few ratings, while OOF prevents each row from carrying its own label information. Here, OOF is a feature-generation step rather than five-fold cross-validation for the entire model [5]. The number of ratings used to compute training rates differs from that used for validation/test; this distributional difference remains a limitation when interpreting results.

6 Algorithms, Baseline, and Evaluation Criteria

6.1 Model Configurations

Table 8 lists the four classification algorithms evaluated in the notebook. Each pipeline has a separately cloned preprocessor and estimator, preventing fitted state from a previous experiment from being overwritten. The algorithm seed is 42.

Table 8: Configurations held fixed across the five feature groups.

Algorithm	Main configuration
Logistic Regression (LR)	class_weight=balanced, max_iter=1000
Decision Tree (DT)	class_weight=balanced, max_depth=20, min_samples_leaf=50
Linear SVM (SVM)	LinearSVC, class_weight=balanced, max_iter=3000
Neural Network (MLP)	Two hidden layers (64, 32), ReLU, Adam; alpha=0.001; batch=1024; learning_rate_init=0.001; max. 100 epochs; tol=0.001; n_iter_no_change=5; early_stopping=False

LR and Linear SVM provide linear classifiers; DT can represent nonlinear splits through condition-based rules; and MLP learns nonlinear transformations through hidden layers. The project does not perform exhaustive hyperparameter optimization. The three algorithms that support `class_weight` use the balanced setting; MLP retains default sample weights, so the algorithms do not handle class imbalance identically.

MLP stopping is based on training-loss improvement, without an internal holdout split. This avoids using labels from an internal holdout to create target encodings for the fit subset. Loss-based stopping does not directly optimize Macro F1. Configurations remain fixed and are not changed based on test results.

6.2 Baseline

We fit a `DummyClassifier` with the `most_frequent` strategy before the other algorithms. For this dataset, the baseline always predicts class 1. On validation and test, it achieves about 57.62% Accuracy and a class 1 F1 of about 0.7311, but class 0 Recall is 0, Macro F1 is only about 0.3656, and ROC-AUC is 0.5. This reference point helps identify models whose aggregate scores are driven by predicting the majority class.

6.3 Metrics and Selection Rule

Class 1 is designated as positive: TP is a correctly predicted high rating, FP is a high-rating prediction for an actual score from 1 to 7, FN is a missed actual score from 8 to 10, and TN is a correct prediction for the 1–7 group. Metrics are computed consistently with scikit-learn [3]:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \tag{3}$$

$$\text{Precision}_1 = \frac{TP}{TP + FP}, \qquad \text{Recall}_1 = \frac{TP}{TP + FN}, \tag{4}$$

$$\text{Recall}_0 = \frac{TN}{TN + FP}, \qquad F1_k = \frac{2\,\text{Precision}_k\,\text{Recall}_k}{\text{Precision}_k + \text{Recall}_k}, \tag{5}$$

$$\text{Macro F1} = \frac{F1_0 + F1_1}{2}. \tag{6}$$

The ROC curve plots TPR against FPR as the threshold varies; AUC is computed from continuous probabilities or decision scores, not predicted 0/1 labels. AUC measures discrimination across thresholds, not Accuracy or probability calibration at the selected threshold.

The project selects configurations by **validation Macro F1** to identify both classes. Ties are broken by ROC-AUC and then Accuracy. Per-class Precision, Recall, and class 1 F1 describe trade-offs rather than being combined into an arbitrary composite score. The notebook uses the estimators' default decision rules: a probability boundary of 0.5 for binary LR, DT, and MLP, and a decision-score boundary of 0 for Linear SVM. These boundaries are not tuned on the test set. For zero-denominator classification metrics, the implementation returns 0.

7 Validation Results and Model Selection

7.1 Comparison Across Feature Groups

Table 9 and Figure 8 compare the four algorithms using the same metric. Full Accuracy, Precision, Recall, F1, and AUC results for all 20 configurations are provided in Appendix A.

Table 9: Validation Macro F1 for four algorithms across five input feature groups.

Model	Initial	Metadata	Country	Count	Target Encoding
LR	0.5380	0.5552	0.5624	0.5638	0.6597
DT	0.5318	0.5303	0.5491	0.5855	0.6455
SVM	0.5380	0.5555	0.5627	0.5643	0.6594
MLP	0.3668	0.5232	0.5646	0.5653	0.6437

Source: Notebook, selection summary in Section 5.6.

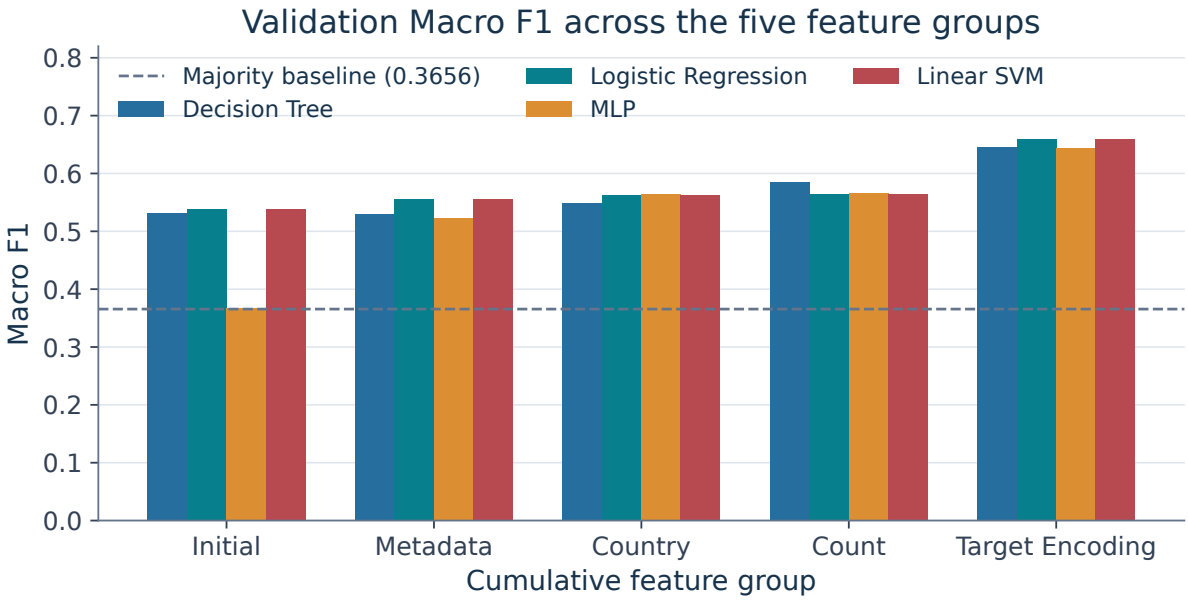


Figure 8: Validation Macro F1 by algorithm and feature group; the dashed line is the baseline.
Source: Notebook, Section 5.6.

With the Initial feature group, age, year, and missingness indicators provide limited classification ability. LR and SVM have Macro F1 of about 0.5380 but Accuracy of about 0.5550, below the baseline's Accuracy. This is not a contradiction: the models identify additional class 0 ratings at the cost of some class 1 ratings. Initial MLP has class 1 Recall near 1 but class 0 Recall of only about 0.0013, yielding Macro F1 of just 0.3668. These cases illustrate why configurations should not be selected using Accuracy or class 1 F1 alone.

Metadata increases Macro F1 and AUC for LR/SVM, but does not improve every model or metric. For example, DT Macro F1 decreases slightly from 0.5318 to 0.5303. After adding Country, all four algorithms improve on Metadata in this split; the gain is larger for MLP than for the other models. These observations compare each algorithm with itself while its configuration is held fixed.

7.2 Effects of Count Features and Target Encoding

Count features increase DT Macro F1 from 0.5491 to 0.5855. The gains are smaller for LR/SVM. Although the Count group improves Macro F1 over Country, class 0 Recall decreases for some models; therefore, not every type of error improves.

Adding Target Encoding produces the clearest gains for LR/SVM among the tested groups. For LR, Macro F1 increases by about 0.0958 over Count, while Recall increases for both classes. Because only two ID-based rates are added at this step, the results provide predictive evidence in this split that rating tendencies from other training records add value. They do not measure causal effects of users or books, nor do they guarantee that the gain will persist under an ID-based or temporal split.

Table 10: Logistic Regression performance as input feature groups are added sequentially.

Step	Accuracy	Macro F1	Δ Macro F1	AUC
Initial → Metadata	0.5578	0.5552	+0.0172	0.5879
Metadata → Country	0.5653	0.5624	+0.0072	0.5971
Country → Count	0.5670	0.5638	+0.0014	0.6022
Count → Target Encoding	0.6610	0.6597	+0.0958	0.7338

Source: Notebook, delta tables; deltas computed before rounding.

Table 11: Validation metrics for the Target Encoding feature group.

	Acc.	Prec. 1	Recall 0	Recall 1	Macro F1	AUC
LR	0.6610	0.7443	0.7071	0.6271	0.6597	0.7338
SVM	0.6607	0.7445	0.7080	0.6259	0.6594	0.7332
DT	0.6480	0.7207	0.6653	0.6352	0.6455	0.7154
MLP	0.6581	0.6875	0.5393	0.7455	0.6437	0.7116

Source: Notebook, Section 5.5.4.

7.3 Configuration Selected by Validation

In the supplied notebook execution, all 20 model–feature-group configurations are ranked by validation Macro F1, ROC-AUC, and then Accuracy before the current test-evaluation cell is run. **Logistic Regression + Target Encoding** ranks first with a Macro F1 of **0.6597**, Accuracy of **66.10%**, and ROC-AUC of **0.7338**. Class 1 Precision is 0.7443; class 0 and class 1 Recall are 0.7071 and 0.6271, respectively.

SVM + Target Encoding is close, with a Macro F1 of about 0.6594. The difference from LR is small, so LR can be said to lead under the selected criterion in this split only; no significance test or multi-seed evaluation establishes a stable advantage. LR is retained for test evaluation without refitting on the test set or reselecting the model based on test results.

8 Test Evaluation and Generalization Scope

8.1 Selected Model Compared with the Baseline

The pipeline fitted on the 70% training partition and selected by validation is evaluated unchanged. Keeping this fitted pipeline preserves the experiment's declared protocol. A later refit on the combined training and validation partitions would require recomputing count tables, OOF features, and preprocessing parameters; its results would belong to a separate fitted model. Table 12 reports results for 65,051 test ratings.

Table 12: Test results for Logistic Regression + Target Encoding and the baseline.

Metric	Selected model	Baseline
Accuracy	0.6565	0.5762
Class 1 Precision	0.7407	0.5762
Class 0 Recall	0.7041	0.0000
Class 0 F1	0.6347	0.0000
Class 1 Recall	0.6215	1.0000
Class 1 F1	0.6758	0.7311
Macro F1	0.6553	0.3656
ROC-AUC	0.7306	0.5000

Source: Notebook, Section 5.7; baseline fit on training data only.

Accuracy increases by 8.03 percentage points, Macro F1 by 0.2897, and ROC-AUC by 0.2306 over the baseline. Class 0 Recall increases from 0 to 0.7041, indicating that the model identifies many ratings in the 1–7 group. In exchange, class 1 Recall is lower than for the baseline that always predicts class 1; class 1 F1 also decreases from 0.7311 to 0.6758, so the gain is not uniform across metrics.

Test Macro F1 is 0.6553, only 0.0044 below validation, so the model shows no sign of overfitting the validation split.

8.2 Confusion Matrix and Error Types

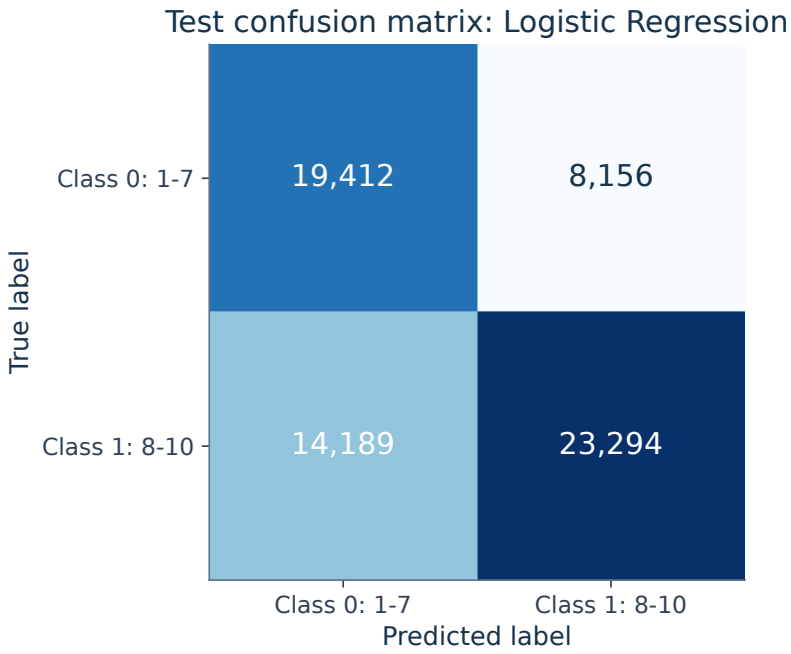


Figure 9: Test confusion matrix; rows show true labels and columns show predicted labels.
Source: Notebook, Section 5.7.

Read Figure 9 in the order of labels 0 and 1:

- **TN = 19,412:** actual 1–7 ratings correctly predicted as class 0.
- **FP = 8,156:** actual 1–7 ratings incorrectly predicted as high ratings.
- **FN = 14,189:** actual 8–10 ratings missed and predicted as class 0.
- **TP = 23,294:** actual 8–10 ratings correctly identified.

Of 31,450 ratings predicted as high, 23,294 are correct, corresponding to a class 1 Precision of about 74.07%. The model identifies about 62.15% of the 37,483 actual high ratings. Its class 0 F1 is 0.6347, so Macro F1 is $(0.6347 + 0.6758)/2 \approx 0.6553$. If the priority is to find more high ratings, a threshold change could be investigated on development data; if the priority is higher precision among high-rating predictions, the cost of FP errors should be evaluated. The default threshold of 0.5 is kept here.

8.3 ROC and Discrimination Across Thresholds

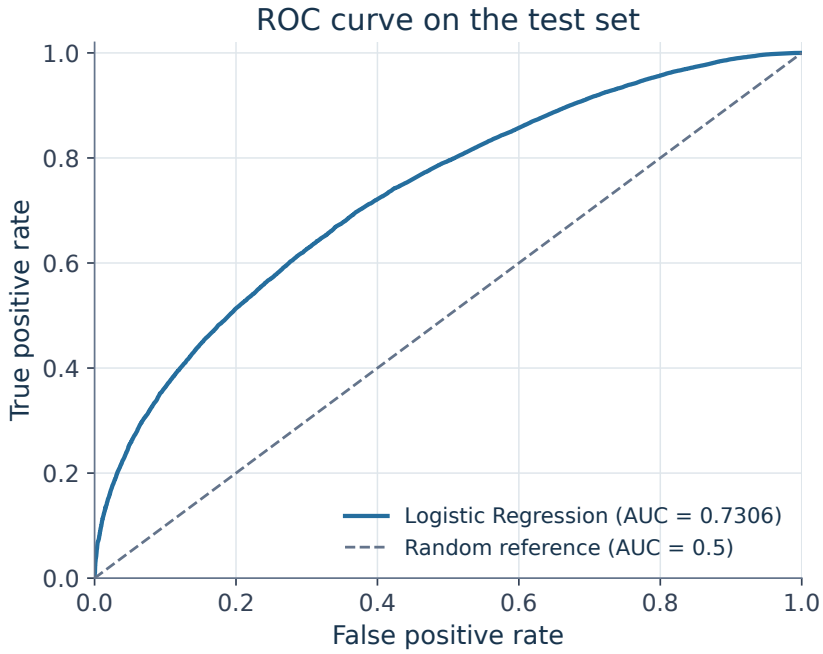


Figure 10: ROC curve for the final model on the test set.
Source: Notebook, Section 5.8.

The ROC curve lies above the reference diagonal, and AUC is 0.7306, higher than the baseline value of 0.5. This provides evidence of the model's ability to discriminate between the two classes using continuous prediction scores rather than a single threshold (AUC measures ranking, not calibration).

8.4 Results by Seen and Unseen IDs

An ID is considered "new" if it has no ratings in the training data used to fit the model. These segments are reported only after model selection and are not used to select the model again.

Table 13: Test results by whether users and books appeared in the training set.

Segment	Ratings	Acc.	Recall 0	Recall 1	Macro F1	AUC
Seen user and book	37,143	0.6644	0.6697	0.6609	0.6589	0.7320
New user, seen book	5,513	0.5674	0.6677	0.4963	0.5673	0.6118
Seen user, new book	19,908	0.6832	0.7390	0.6361	0.6832	0.7632
New user and book	2,487	0.5219	0.9260	0.1097	0.4234	0.5935

Source: Notebook, Section 5.7; the four segments sum to 65,051 ratings.

The seen-user/seen-book segment contains 37,143 ratings and has a Macro F1 of 0.6589. When both IDs are new, Macro F1 falls to 0.4234 and class 1 Recall is 0.1097. In this segment, both target-encoding features fall back to the training prior and both count features are 0, leaving only metadata, age, country, and missingness information to distinguish records. This result cautions against generalizing aggregate performance to all new-user/new-book cases.

The seen-user/new-book segment achieves a Macro F1 of 0.6832, higher than the segment where both are seen; since the segments differ in composition and size, this comparison is descriptive only.

9 Discussion, Applications, and Limitations

9.1 Interpretation of Findings

The EDA shows substantial missingness in age, an uneven distribution of rating counts, and a difference between popularity and mean score. Basic demographic and metadata features do not provide strong discrimination. In these experiments, ID-based rating-tendency statistics produce the clearest gains for LR/SVM. This is predictive value under the current design, not evidence that any individual feature causes rating behavior.

For applications, the model could support estimates of high-rating likelihood when user/book information and other ratings are available for lookup. Precision and Recall help inform trade-offs for a future objective. However, before using the model to rank recommendations, a ranking-evaluation design and data on candidate books that have not been rated are needed. The current metrics do not establish purchasing benefit, revenue, or business impact.

9.2 Limitations to Consider

1. **Missingness and data quality:** age, publication year, and book metadata have substantial missingness. Imputation and missingness indicators do not recover the original information. Age/year thresholds are conventions that may mark some valid values as missing; country parsed from Location may remain incorrect or inconsistent.
2. **Rating-level split:** the same users and books can occur across multiple splits. The data have no timestamps to establish that all statistics would be available before a rating in a real deployment. The results are not a pure cold-start or future-prediction evaluation.
3. **Label scope:** excluding zero-valued ratings matches the explicit-rating task definition but limits conclusions to records scored from 1 to 10. This group may differ from the population of users or books that have not been rated.
4. **Statistical inference:** EDA uses the full sample, and ratings may be dependent by user or book. User-level rates have different precision because users contribute different numbers of ratings; small p-values are not used to claim causality or stable population-level differences.
5. **Selection and independence:** all 20 configurations share one validation split and one seed, and hyperparameters are not exhaustively optimized. The test set was inspected in earlier project versions, so the current evaluation is not a fully independent post-development confirmation.

6. **Distribution differences:** training uses OOF rates and counts with the current rating removed, while validation/test use statistics from all training data. Smoothing reduces but does not eliminate these differences. MLP stops based on training loss rather than directly optimizing Macro F1, and class balancing differs across algorithms.

10 Conclusion and Future Work

10.1 Conclusion

The project covers the full workflow from relational data validation, cleaning, EDA, and label definition to feature engineering, training, and evaluation. The classification dataset contains 433,671 ratings. Retaining ISBNs as strings, normalizing keys, preserving ratings without metadata, and fitting preprocessing on training data are important for consistency.

Among the 20 evaluated configurations, **Logistic Regression + Target Encoding** is selected by validation Macro F1. On test, the model achieves **65.65% Accuracy, 0.6553 Macro F1, and 0.7306 ROC-AUC**. Compared with the baseline, it improves Accuracy and the equal-weighted summary of both classes, while class 1 Recall and F1 decrease. Thus the gain in Macro F1 reflects recovery of class 0 rather than an improvement on every measure. Weaker results when both IDs are new show why performance should be considered by use case rather than summarized by one aggregate score.

10.2 Next Steps

1. **Validate on new data:** evaluate on an untouched holdout and additional seeds or cross-validation. Refit preprocessing, count features, and OOF encodings within each fold.
2. **Match the generalization design:** test user- or book-based splits, or time-based splits if timestamps become available; report performance by unseen-ID status and history density.
3. **Optimize for the intended objective:** tune hyperparameters and thresholds on development data; assess PR-AUC, calibration, FP/FN costs, and variability across splits.
4. **Improve representations:** refine country normalization and test book-title text, genres, or descriptions under the same evaluation protocol.
5. **Evaluate recommendations separately:** define candidate books and assess ranking metrics in addition to classification performance.

Reproducibility and Data Provenance

The analysis is implemented in `mid_term_code.ipynb`, available with the data and report source in the project repository: https://github.com/Bluenip/Data_analyst_python_midterm_project. The tables and figures in this report agree with the saved notebook results. All metrics are reported to four decimal places; deltas are calculated before rounding.

To reproduce the analysis, install the packages in `requirements.txt`, place the three CSV files under `DATA_SET/`, and run the notebook from the project root in a fresh kernel. The data-split seed is 56; the model and OOF seeds are 42. Table 14 records the execution environment. Minor numerical differences may occur across library versions or backends.

Table 14: Environment recorded in the notebook run used for this report.

Component	Version
Python	3.14.9
numpy	2.5.4
pandas	3.1.0
matplotlib	3.11.2
scipy	1.18.1
scikit-learn	1.9.1

The supplied notebook has SHA-256 prefix `5a0991ddba42d256`.

References

[1] Möbius. *Book Recommendation Dataset*. Kaggle. https://www.kaggle.com/datasets/arashnic/book-recommendation-dataset.

[2] Möbius. *Practice Recommendation Algorithms with Book Recommendation Dataset*. Description of Users, Books, Ratings, and the meaning of rating scores. https://www.kaggle.com/discussions/general/202108.

[3] Scikit-learn developers. *Metrics and scoring: quantifying the quality of predictions*, section on classification metrics. https://scikit-learn.org/stable/modules/model_evaluation.html.

[4] Scikit-learn developers. *Common pitfalls and recommended practices*, section on data leakage. https://scikit-learn.org/stable/common_pitfalls.html.

[5] Scikit-learn developers. *TargetEncoder*, documentation on cross-fitting, smoothing, and unseen categories. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.TargetEncoder.html.

[6] *mid_term_code.ipynb*. Project source code and experiment outputs. https://github.com/Bluenip/Data_analyst_python_midterm_project.

A Complete Validation Results for 20 Configurations

LR: Logistic Regression; DT: Decision Tree; SVM: Linear SVM; MLP: multilayer perceptron. Acc.: Accuracy; P1: class 1 Precision; R0/R1: class 0/1 Recall; $F1_1$: class 1 F1; MF1: Macro F1; AUC: ROC-AUC. All 20 configurations use the same validation set; this table does not include test results.

Table 15: All validation metrics for five feature groups and four algorithms.

Group	Model	Acc.	P1	R0	R1	$F1_1$	MF1	AUC
Initial	LR	0.5550	0.6066	0.4286	0.6479	0.6266	0.5380	0.5455
Initial	DT	0.5319	0.6229	0.6085	0.4755	0.5393	0.5318	0.5586
Initial	SVM	0.5550	0.6066	0.4286	0.6479	0.6266	0.5380	0.5455
Initial	MLP	0.5764	0.5764	0.0013	0.9993	0.7311	0.3668	0.5516
Metadata	LR	0.5578	0.6341	0.5684	0.5500	0.5891	0.5552	0.5879
Metadata	DT	0.5318	0.6469	0.6937	0.4126	0.5038	0.5303	0.5793
Metadata	SVM	0.5580	0.6345	0.5695	0.5496	0.5890	0.5555	0.5879
Metadata	MLP	0.5896	0.6040	0.2555	0.8353	0.7011	0.5232	0.5976
Country	LR	0.5653	0.6402	0.5717	0.5606	0.5978	0.5624	0.5971
Country	DT	0.5491	0.6475	0.6465	0.4775	0.5496	0.5491	0.5920
Country	SVM	0.5654	0.6407	0.5733	0.5596	0.5974	0.5627	0.5971
Country	MLP	0.5928	0.6246	0.3992	0.7352	0.6754	0.5646	0.6082
Count	LR	0.5670	0.6408	0.5690	0.5655	0.6008	0.5638	0.6022
Count	DT	0.5882	0.6630	0.5990	0.5802	0.6189	0.5855	0.6324
Count	SVM	0.5674	0.6414	0.5703	0.5653	0.6009	0.5643	0.6022
Count	MLP	0.6056	0.6248	0.3554	0.7896	0.6976	0.5653	0.6333
Target Encoding	LR	0.6610	0.7443	0.7071	0.6271	0.6807	0.6597	0.7338
Target Encoding	DT	0.6480	0.7207	0.6653	0.6352	0.6753	0.6455	0.7154
Target Encoding	SVM	0.6607	0.7445	0.7080	0.6259	0.6801	0.6594	0.7332
Target Encoding	MLP	0.6581	0.6875	0.5393	0.7455	0.7153	0.6437	0.7116

Note: Feature groups are ordered by the sequence in which inputs are added, not by final rank. The selected configuration has the highest MF1; ties are broken by AUC and then Acc. Each algorithm's configuration is held fixed across groups, but the project does not establish that any configuration is optimal across all representations, hyperparameters, or data splits.